

# **“Predicting Uber Ride Cancellations: A Comparative Analysis of Pre-Booking and Post-Booking Features”**

## **Introduction**

The dataset provides detailed information on Uber ride bookings in 2024, enabling analysis of booking behavior, cancellation patterns, and operational performances. It includes approximately 150,000 total bookings across multiple vehicle types, with features such as pickup and drop locations, ride timing, customer and driver behavior, and ride outcomes. This dataset allows for both exploratory analysis and predictive modeling of ride cancellations, which is a key operational challenge in ride-sharing platforms.

Our primary objective of this project is to predict whether an Uber ride will be cancelled using booking-related features such as time, location, and vehicle type.

Additionally, we compare model performance under two scenarios:

- Using all available features
- Using only pre-booking features (real-world prediction scenario where the model only knows features that would be present before a booking)

## **Target Variable**

A binary target variable (Cancelled) was created to represent ride outcomes.

- Missing values in the cancellation columns were replaced with 0
- A ride was labeled as
  - 1 (Cancelled) - if either the customer or driver cancelled
  - 0 (Not Cancelled) - otherwise

```

#target variable - Creating a new column (Cancelled) that turns messy, multi-column cancellation info into a single clean binary variable.
#A ride is considered cancelled (1) if either the customer or the driver cancels the ride, otherwise (0) it is not cancelled
#converting the NAN into 0 not canceled
#Cancelled by Customer/Driver = 1 -> Cancelled to deal with all the ambiguity within the data
dfClean['Cancelled Rides by Customer'] = dfClean['Cancelled Rides by Customer'].fillna(0)
dfClean['Cancelled Rides by Driver'] = dfClean['Cancelled Rides by Driver'].fillna(0)

#target variable - np.where is saying if condition is TRUE assign 1 else assign 0
dfClean['Cancelled'] = np.where(
    (dfClean['Cancelled Rides by Customer'] == 1) |
    (dfClean['Cancelled Rides by Driver'] == 1),
    1, 0)

#check
print("How Many Rides Were Cancelled VS Not Cancelled?")
dfClean['Cancelled'].value_counts()

```

| Customer Cancel | Driver Cancel | Result (Cancelled) |
|-----------------|---------------|--------------------|
| 0               | 0             | 0 (Not Cancelled)  |
| 1               | 0             | 1 (Cancelled)      |
| 0               | 1             | 1 (Cancelled)      |
| 1               | 1             | 1 (Cancelled)      |

This approach was chosen to consolidate multiple cancellation indicators into a single interpretable variable for classification. A key challenge in this step was converting missing data into meaningful information that could be used for prediction. By treating missing cancellation values as non-cancellations, we were able to construct a consistent and interpretable binary target variable. We focused on explicit cancellations and excluded operational outcomes such as “No Driver Found” or “Incomplete”. The results from the target variable above proved to be as such.

```

How Many Rides Were Cancelled VS Not Cancelled?
Cancelled
0    112500
1     37500

```

This means that out of 150,000 total bookings 75% of them were not cancelled and 25% were cancelled. We note that this is a slight imbalance but something that did not need to be addressed yet.

## Data Preprocessing

We began preprocessing by creating a copy of the dataset to preserve the original data while performing transformations. Initial analysis focused on identifying missing values and determining their significance.

Columns with more than 80% missing values were removed, as they were considered unreliable for modeling. Additionally, several columns containing post-outcome information (such as cancellation reasons) were dropped to prevent data leakage, ensuring that models would not learn from information unavailable at prediction time.

|                                   |      |
|-----------------------------------|------|
| Incomplete Rides                  | 94.0 |
| Incomplete Rides Reason           | 94.0 |
| Reason for cancelling by Customer | 93.0 |
| Driver Cancellation Reason        | 82.0 |
| Customer Rating                   | 38.0 |
| Driver Ratings                    | 38.0 |
| Payment Method                    | 32.0 |
| Booking Value                     | 32.0 |
| Avg CTAT                          | 32.0 |
| Ride Distance                     | 32.0 |
| Avg VTAT                          | 7.0  |
| Date                              | 0.0  |
| Time                              | 0.0  |
| Booking ID                        | 0.0  |
| Vehicle Type                      | 0.0  |
| Customer ID                       | 0.0  |
| Booking Status                    | 0.0  |
| Pickup Location                   | 0.0  |
| Cancelled Rides by Driver         | 0.0  |
| Cancelled Rides by Customer       | 0.0  |
| Drop Location                     | 0.0  |
| Cancelled                         | 0.0  |

Further analysis revealed that missing values were not random. For features such as Ride Distance, Booking Value, and Ratings, missing values occurred primarily when rides were cancelled. This indicates that the absence of data represents a meaningful outcome (i.e., the ride did not occur), rather than incomplete data collection.

To preserve this meaning:

- Numerical features related to completed rides were filled with 0
- Categorical features such as Payment Method were filled with “Unknown”
- Remaining numerical features (Avg CTAT, Avg VTAT) were filled using the median to reduce the impact of outliers

```
#officially handling the missing values caused by cancellation
#booking value, ride distance, customer rating, driver ratings have missing values
#0 value 0 rating means the ride never happened so this is what we are encoding
dfClean['Booking Value'] = dfClean['Booking Value'].fillna(0)
dfClean['Ride Distance'] = dfClean['Ride Distance'].fillna(0)
dfClean['Customer Rating'] = dfClean['Customer Rating'].fillna(0)
dfClean['Driver Ratings'] = dfClean['Driver Ratings'].fillna(0)

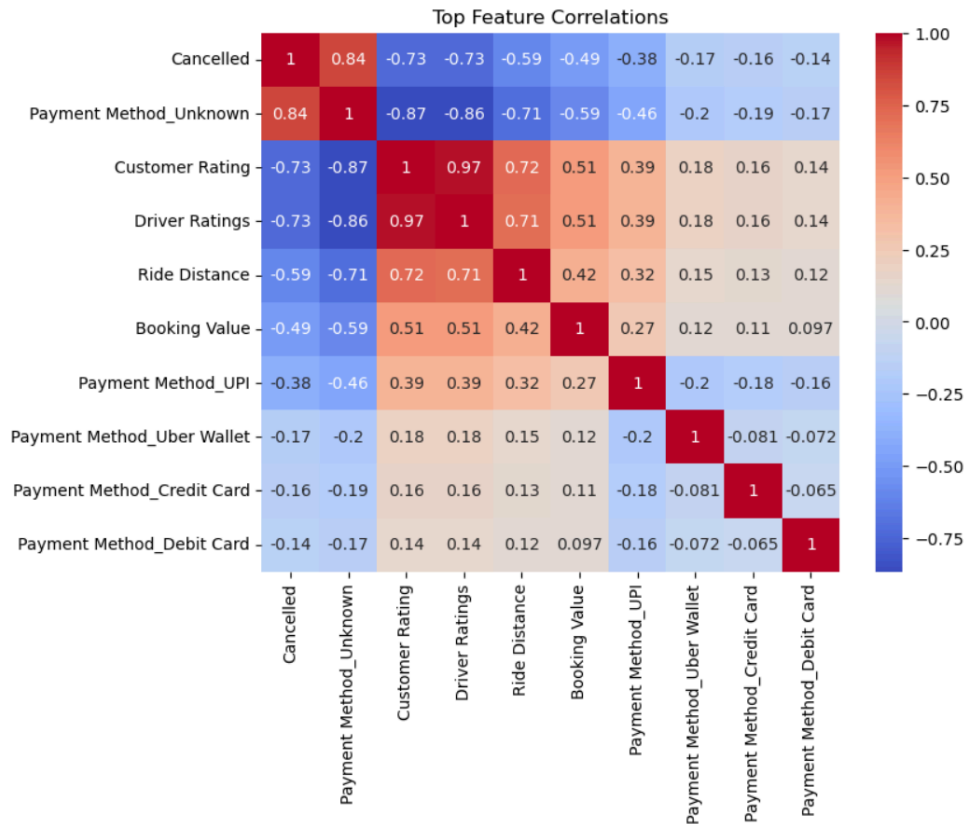
#handling the categorical missing values
#no ride = no payment method = unknown to preserve this to prevent model cheating
dfClean['Payment Method'] = dfClean['Payment Method'].fillna('Unknown')

#handling true numeric missing values
#using median since median isnt sensitive to outliers
#these values were missing due to noise and dealing with them by filling with median
dfClean['Avg CTAT'] = dfClean['Avg CTAT'].fillna(dfClean['Avg CTAT'].median())
dfClean['Avg VTAT'] = dfClean['Avg VTAT'].fillna(dfClean['Avg VTAT'].median())
```

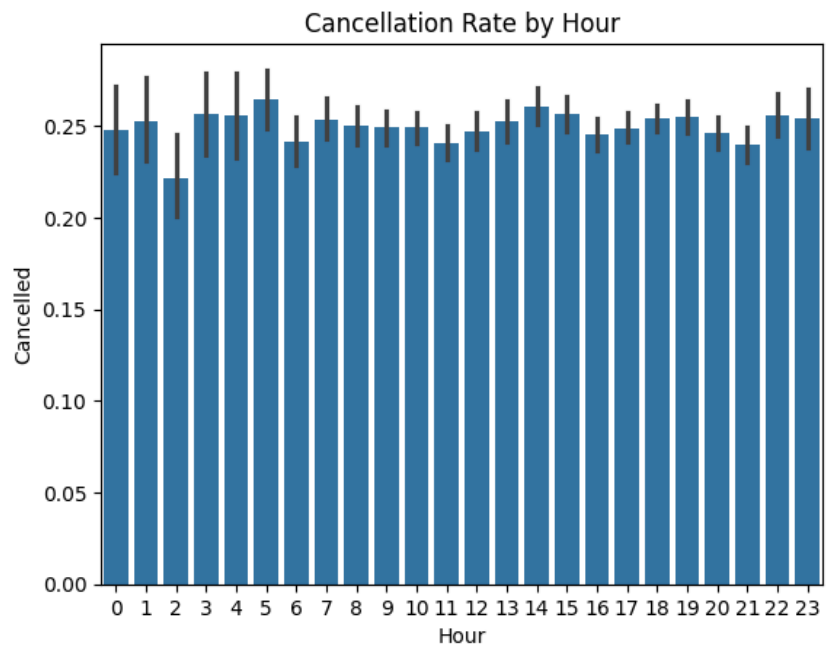
This approach ensured that missing data was handled in a way that maintained real-world interpretability rather than applying uniform imputation. After cleaning and preparing the dataset, exploratory data analysis was conducted to identify patterns and relationships that could inform model selection.

## **EDA**

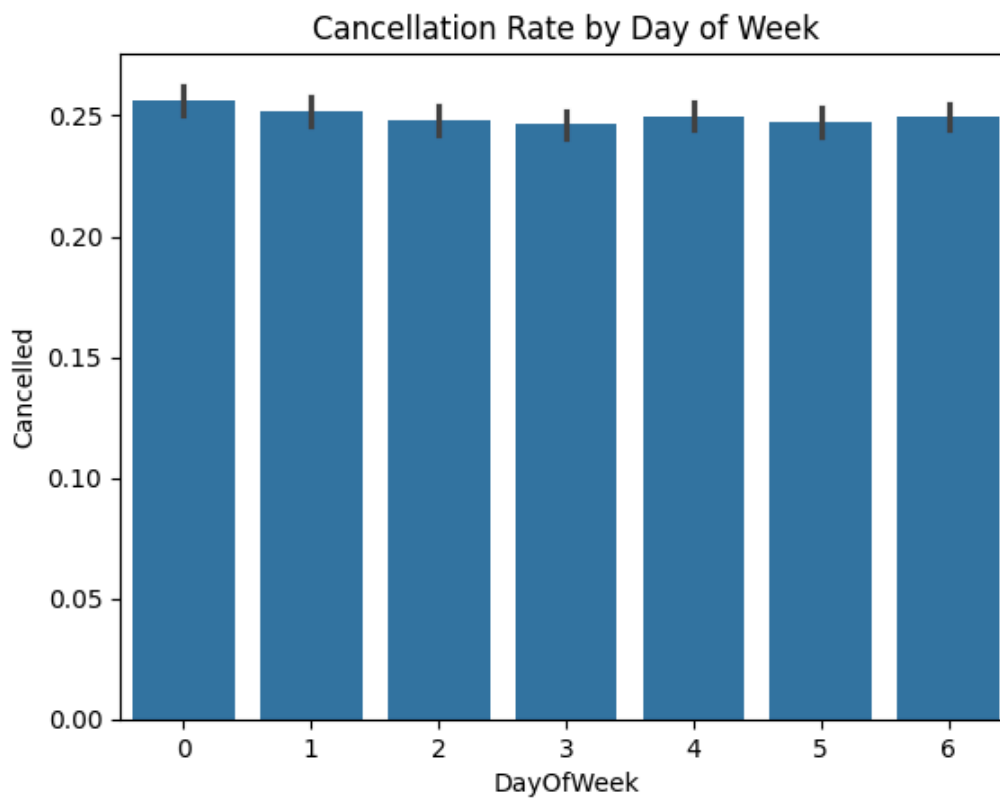
Exploratory Data Analysis was conducted to identify patterns, relationships, and key factors influencing ride cancellations. Visualizations and correlation analysis were used to better understand how different features relate to the target variable “Cancelled”. Using a correlation matrix, we ensured that there are no correlation issues. Notably, driver rating and customer rating has a strong correlation as they would be tied together for how one views the other. Other strong negative correlations between cancellation and the features were: Ride Distance and Booking Value. These four features indicate that rides with higher ratings, longer distances, and higher booking values are significantly less likely to be cancelled. We also noted a strong positive correlation was observed with Payment Method. This suggested that cancelled rides are often associated with missing or undefined payment information. This suggests that cancelled rides are often associated with missing or undefined payment information. By enforcing prepayment you reduce the risk of losing a ride to being cancelled.



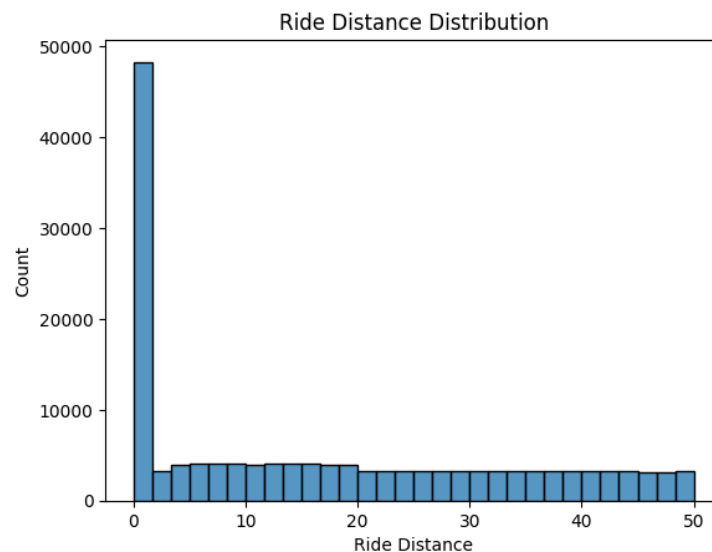
Furthermore, we took the time to determine patterns by Time, Day of Week, as well as ride distance distribution, and vehicle type analysis. In the analysis of cancellations by hour it showed that cancellation rates remained relatively consistent throughout the day. There were no strong peaks or drops observed. This indicates that time of day is not a major factor influencing ride cancellations in this dataset.



Similarly, analyzing cancellation patterns by day of week showed minimal variation in cancellation rates across different days. Which suggests that ride cancellations are not strongly dependent on the day of the week either.



The Ride Distance Distribution revealed a large concentration of rides at 0 distance. Ride Distance is measured in kilometers, what we were able to understand is that many rides were cancelled before they began, resulting in no recorded distance. The distribution is highly skewed with most completed rides falling within a moderate distance range. Which suggests that a potential log transformation could help with the skewness. This also offers promising insights for ride share companies to find a way to mitigate this issue from consistently happening.



By analyzing vehicle types it suggested that cancellation rates were relatively similar across all vehicle types. This suggests that vehicle type does not significantly influence whether a ride will be cancelled or not.

From the analysis several important insights were identified:

- Cancellation is strongly associated with ride completion-related features such as distance, value, and range.



- Temporal features had very minimal predictive power
- Cancellation behavior is not heavily dependent on vehicle type.

## Model Performance

|   | Model               | Accuracy | Precision | Recall   | F1 Score |
|---|---------------------|----------|-----------|----------|----------|
| 3 | Random Forest       | 0.997400 | 1.000000  | 0.989600 | 0.994773 |
| 5 | XGBoost             | 0.997400 | 1.000000  | 0.989600 | 0.994773 |
| 6 | Gradient Boosting   | 0.997400 | 1.000000  | 0.989600 | 0.994773 |
| 0 | Logistic Regression | 0.929500 | 0.780031  | 1.000000 | 0.876424 |
| 1 | Naive Bayes         | 0.929500 | 0.780031  | 1.000000 | 0.876424 |
| 4 | SGD Classifier      | 0.926467 | 0.779869  | 0.983467 | 0.869914 |
| 2 | KNN                 | 0.744567 | 0.479569  | 0.255067 | 0.333014 |

Using python, we determined the most accurate models that can be applied for the dataset and had them ranked based on the f1-score. This was done to find the features that most impacted the dataset and to ensure there is no overfitting in the model. For each model we used a confusion matrix along with printing the precision, recall, f1-score, support, and accuracy values and then printed the top features to determine how accurate the model is in predicting, how many false positives and negatives there are, and the top features of the model. We want to ensure that throughout the models that the same features are highlighted to ensure there is no variability.

|   | Model               | Accuracy | Precision | Recall   | F1 Score |
|---|---------------------|----------|-----------|----------|----------|
| 6 | Gradient Boosting   | 0.995633 | 0.992910  | 0.989600 | 0.991252 |
| 5 | XGBoost             | 0.995633 | 0.992910  | 0.989600 | 0.991252 |
| 3 | Random Forest       | 0.995600 | 0.992909  | 0.989467 | 0.991185 |
| 1 | Naive Bayes         | 0.889933 | 0.776257  | 0.786400 | 0.781296 |
| 2 | KNN                 | 0.690533 | 0.259439  | 0.128267 | 0.171663 |
| 4 | SGD Classifier      | 0.749533 | 0.062500  | 0.000133 | 0.000266 |
| 0 | Logistic Regression | 0.750000 | 0.000000  | 0.000000 | 0.000000 |

Moreover we want to ensure no data leakage so we also cleaned the dataset to only show the pre-booking and then based on that determined which three models to use as comparison with each other. Based on the results, we used these predictive models: XGBoost, KNN, and logistic regression.

XGBOOST - FULL DATASET

[[22500 0]

[ 78 7422]]

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 22500   |
| 1            | 1.00      | 0.99   | 0.99     | 7500    |
| accuracy     |           |        | 1.00     | 30000   |
| macro avg    | 1.00      | 0.99   | 1.00     | 30000   |
| weighted avg | 1.00      | 1.00   | 1.00     | 30000   |

XGBoost is a gradient boosting algorithm that is a combination of multiple algorithms to ensure a high performance model. It utilizes decision trees as a base learner and builds off it so it will correct any error from past ones. It will repeat the process until a criterion is met. The model had an accuracy of 1. Based on the confusion matrix there were no false positives and minimal false negatives with 78. Although the model shows a macro F1 score of 1 which could indicate that the model is overfitting.

```
XGBOOST - PRE-BOOKING DATASET
```

```
Confusion Matrix:
```

```
[[22447    53]
 [   78  7422]]
```

```
Classification Report:
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 22500   |
| 1            | 0.99      | 0.99   | 0.99     | 7500    |
| accuracy     |           |        | 1.00     | 30000   |
| macro avg    | 0.99      | 0.99   | 0.99     | 30000   |
| weighted avg | 1.00      | 1.00   | 1.00     | 30000   |

With the pre-booking dataset the model still showed an accuracy of 1 with again notable few false negatives. There is now an increase of false positives now with 53.

KNN - FULL DATASET

```
[[20424 2076]
```

```
[ 5587 1913]]
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.79      | 0.91   | 0.84     | 22500   |
| 1            | 0.48      | 0.26   | 0.33     | 7500    |
| accuracy     |           |        | 0.74     | 30000   |
| macro avg    | 0.63      | 0.58   | 0.59     | 30000   |
| weighted avg | 0.71      | 0.74   | 0.71     | 30000   |

K-nearest neighbor algorithm (KNN) is a machine learning algorithm that identifies the closest data points from the data set and predicts based on the average value of the neighbors.

The k value is determined based on cross validation from other models to ensure that the model has the correct amount of neighbors for the algorithm to look at and to not underfoot or overfit.

The model had an accuracy of 0.74. Based on the confusion matrix there were significant false negatives indicating the model had missed actual cancellations. The imbalance of the confusion matrix implies that the model could be favoring the majority class. The model could be improved upon by tuning the k value so it can capture the minority patterns more efficiently.

---

#### KNN - PRE-BOOKING DATASET

Confusion Matrix:

```
[[19754 2746]
 [ 6538  962]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.75      | 0.88   | 0.81     | 22500   |
| 1            | 0.26      | 0.13   | 0.17     | 7500    |
| accuracy     |           |        | 0.69     | 30000   |
| macro avg    | 0.51      | 0.50   | 0.49     | 30000   |
| weighted avg | 0.63      | 0.69   | 0.65     | 30000   |

In comparison, the pre-booking dataset found that the accuracy score had dropped significantly to 0.69. The confusion matrix showed that there is still a notable imbalance.

#### LOGISTIC REGRESSION - FULL DATASET

```
[[20385 2115]
 [    0 7500]]
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 0.91   | 0.95     | 22500   |
| 1            | 0.78      | 1.00   | 0.88     | 7500    |
| accuracy     |           |        | 0.93     | 30000   |
| macro avg    | 0.89      | 0.95   | 0.91     | 30000   |
| weighted avg | 0.95      | 0.93   | 0.93     | 30000   |

Logistic regression is an algorithm that uses continuous values to help predict the odds that an input belongs to a particular class. It uses sigmoid function to convert inputs into a value

between 0 and 1. The accuracy of the logistic regression for the full dataset was 0.93. The model was good at predicting with the full dataset. Notably it had no false negatives which implies that the model correctly identified all actual cancelled rides.

#### LOGISTIC REGRESSION - PRE-BOOKING DATASET

Confusion Matrix:

```
[[12006 10494]
 [ 3701  3799]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.76      | 0.53   | 0.63     | 22500   |
| 1            | 0.27      | 0.51   | 0.35     | 7500    |
| accuracy     |           |        | 0.53     | 30000   |
| macro avg    | 0.52      | 0.52   | 0.49     | 30000   |
| weighted avg | 0.64      | 0.53   | 0.56     | 30000   |

Using the pre-booking dataset, the accuracy of the model was considerably worse with a f1-score of 0.53. Moreover the matrix has an increase of false negatives. This is caused by the lower recall.

## Model Comparison

A feature importance analysis was conducted using correlation with the target variable to better understand differences in model performances between the fully cleaned dataset and the pre-booking dataset that we completed. In the full dataset, the most influential features included Payment Method, Customer Rating, Driver Ratings, Ride Distance, and Booking Value. These features showed strong correlation with the target variable with values as high as 0.84. These features are directly associated with ride completion and are only available after the ride occurs.

For example, cancelled rides often lack payment information, ratings, and distance values, leading to strong predictive signals. This explains why models trained on the full dataset achieved near-perfect performance as they were effectively learning post-outcome indicators. In contrast, the pre-booking dataset shows significantly weaker correlations with the target variable. The most important features such as Avg VTAT and Avg CTAT had correlations below 0.1. This indicates that features available before the ride begins provide limited predictive power for cancellation. This comparison reveals that the strongest predictors of cancellation remain in the full database and are not available at booking time. These insights share very valuable information that when restricted to pre-booking features, the prediction task of determining what might cancel rides becomes significantly more challenging. We confirmed that the high model performance observed was due to data leakage as there are no strong signs to indicate whether the ride will be cancelled or not.

| Top Features - FULL DATASET |          |
|-----------------------------|----------|
| Payment Method_Unknown      | 0.841625 |
| Customer Rating             | 0.728064 |
| Driver Ratings              | 0.727336 |
| Ride Distance               | 0.593727 |
| Booking Value               | 0.494658 |
| Payment Method_UPI          | 0.383426 |
| Payment Method_Uber Wallet  | 0.172371 |
| Payment Method_Credit Card  | 0.156024 |
| Payment Method_Debit Card   | 0.139187 |
| Avg VTAT                    | 0.072891 |

| Top Features - PRE-BOOKING DATASET |          |
|------------------------------------|----------|
| Avg VTAT                           | 0.072891 |
| Avg CTAT                           | 0.018693 |
| Drop Location_Vasant Kunj          | 0.011084 |
| Drop Location_Gurgaon Sector 29    | 0.008369 |
| Pickup Location_Vinobapuri         | 0.007972 |
| Pickup Location_Akshardham         | 0.007896 |
| Drop Location_Panchsheel Park      | 0.007123 |
| Drop Location_Keshav Puram         | 0.006451 |
| Pickup Location_Subhash Chowk      | 0.006376 |
| Drop Location_Bhikaji Cama Place   | 0.006296 |

## Conclusion

This project demonstrates that machine learning models can be highly effective in predicting ride cancellations when using advanced methods. Among all models tested XGBoost consistently achieved the highest performance, while Logistic Regression provided a strong baseline KNN struggled due to the high dimensionality of the dataset. The models had their own

limitations, notably the computation time as the more accurate model took longer to run through ensuring more accuracy. The KNN model needed a low accuracy value for KNN indicates that the model used needs further k values in order to be more accurate. While near-perfect accuracy was achieved using the fully cleaned dataset, further analysis revealed that this performance was largely driven by features that are only available after ride completion. Features such as booking value, ride distance, customer ratings, and payment method introduced data leakage, allowing models to learn from post-outcome information rather than true predictive signals.

To address this, a pre-booking dataset was created using only features available before the ride begins. When models were trained on this dataset, performance decreased significantly, particularly for simpler models such as Logistic Regression and KNN. This highlights the difficulty of predicting cancellations using only pre-ride information.

Feature importance analysis further supported these findings, showing that the strongest predictors in the full dataset were not present in the pre-booking dataset. This confirms that cancellation prediction is largely dependent on information generated after the ride outcome, rather than beforehand.

Overall, this project demonstrates that while machine learning can effectively identify cancellations after they occur, predicting them in advance remains a challenging problem. These findings provide valuable insight into the limitations of predictive modeling in real-world ride-sharing scenarios and highlight the importance of careful feature selection to avoid misleading results.